LYCEUM OF THE PHILIPPINES UNIVERSITY – CAVITE

College of Information Technology and Computer Studies

Application Development — Midterm Laboratory Practical

LPUtility

Network Diagnostics, Infrastructure Intelligence & Academic Utilities
Responsive Web Deployment & Android WebView Delivery

BSIT301NS PRACTICAL REPORT

System Name:	LPUtility (v2.0.0)
Integrated Live APIs:	1. Globalping REST API (jsDelivr) • 2. RIPE Stat Data API (RIPE NCC)
GitHub Repository:	https://github.com/ruru-o/lputility-webview.git
Deployed Web Application:	https://ruru-o.github.io/lputility-webview/
Android Package:	com.lpu.netpulse (LPUtility WebView Application)

October 2026 • Lyceum of the Philippines University • Academic Laboratory Submission

1. Project Overview & Architecture

LPUtility is a web application and native Android WebView application designed for IT and Computer Science students, network technicians, and developers. It provides multi-region network diagnostics, BGP routing intelligence, subnet calculations, and cryptographic utilities in one responsive, high-performance interface.

1.1 Problem Addressed

The initial prototype relied on local PowerShell scripts and third-party package links without real-time telemetry. In this release, LPUtility integrates two complementary live APIs to provide actionable network diagnostics:

- **Multi-Region Latency & Reachability:** Testing targets from Europe, Asia, North America, and Oceania using distributed probes.
- **Autonomous System & BGP Context:** Identifying the authoritative Autonomous System Number (ASN), organization owner, and BGP prefix block.
- **Offline Protocol Utilities:** Interactive IPv4 VLSM subnetting, 128-bit IPv6 RFC 5952 calculations, and NIST SP 800-63B password/key generation.

2. Dual-API Integration

LPUtility integrates two live, production-grade APIs that work together to provide complete network diagnostic information:

API	Provider	Endpoints	Data Provided	Access Method
Globalping API	jsDelivr	`POST /v1/measurements` `GET /v1/measurements/{id}`	Multi-continent ping, DNS, HTTP, and traceroute latency, packet loss, and jitter.	Public REST API, open CORS (*), no client secrets.
RIPE Stat API	RIPE NCC	`GET /data/network-info/` `GET /data/as-overview/`	Authoritative ASN, organization holder, BGP prefix, and allocation registry.	Public RIR data API, open CORS (*), no API key needed.

2.1 API Synergy

When a diagnostic test is triggered for a target (e.g. `1.1.1.1` or `google.com`), LPUtility dispatches active probes across multiple continents through Globalping while concurrently querying RIPE Stat for BGP routing information. The user sees active round-trip measurements alongside the network owner and BGP prefix block.

2.2 Code Implementation (from `src/services/telemetryAggregator.js`)

```
export async function runCombinedTelemetry({ type = 'ping', target, locations }) {
  // Concurrently dispatch Globalping active probes and RIPE Stat BGP routing lookup
  const globalpingPromise = executeGlobalpingTest({ type, target, locations });
  const ripestatPromise = fetchInfrastructureTelemetry(target);

  const [globalpingResult, ripestatResult] = await Promise.all([
    globalpingPromise.catch(err => null),
    ripestatPromise.catch(err => null)
  ]);

  return { target, type, globalping: globalpingResult, ripestat: ripestatResult };
}
```

3. Responsive Design & Media Queries

The application adapts cleanly across desktop, tablet, and mobile viewports down to 320px width using clean CSS Flexbox, CSS Grid, and media queries in `src/styles/layout.css`:

```
/* Tablet & Mobile Breakpoint (<= 768px) */
@media (max-width: 768px) {
  .mobile-menu-btn { display: inline-flex; }
  .desktop-toggle-btn { display: none; }
  .winui-sidebar {
    position: fixed; top: 50px; bottom: 0; left: 0;
    width: 240px; transform: translateX(-100%);
    z-index: 150; box-shadow: 4px 0 20px rgba(0, 0, 0, 0.7);
  }
  .winui-sidebar.mobile-open { transform: translateX(0); }
  .main-content { padding: 12px 12px 50px 12px; }
  .results-grid { grid-template-columns: 1fr; }
}

/* Small Smartphone Viewport (<= 480px) */
@media (max-width: 480px) {
  .target-input-row { flex-direction: column; }
  .run-btn { width: 100%; }
  .summary-card { flex-direction: column; align-items: stretch; }
  .tools-grid { grid-template-columns: 1fr; }
}
```

4. Android Native WebView Implementation

The Android Studio project is located in `responsive webview/`, built with Kotlin for SDK 35 (Android 15):

- **Back Navigation:** Modern `OnBackPressedCallback` checks `webView.canGoBack()` to navigate through browser history before exiting.
- **URL Routing:** Application routes and local assets stay inside WebView; external links open in the device browser via `Intent.ACTION_VIEW`.
- **Progress & Refresh:** Custom `WebChromeClient` updates the horizontal progress bar; `SwipeRefreshLayout` enables pull-to-refresh.
- **Offline Bundle:** Bundled web assets at `app/src/main/assets/web/` ensure full functionality even when offline.

5. Application Screenshots & Evidence

5.1 Desktop Network Diagnostics Workspace

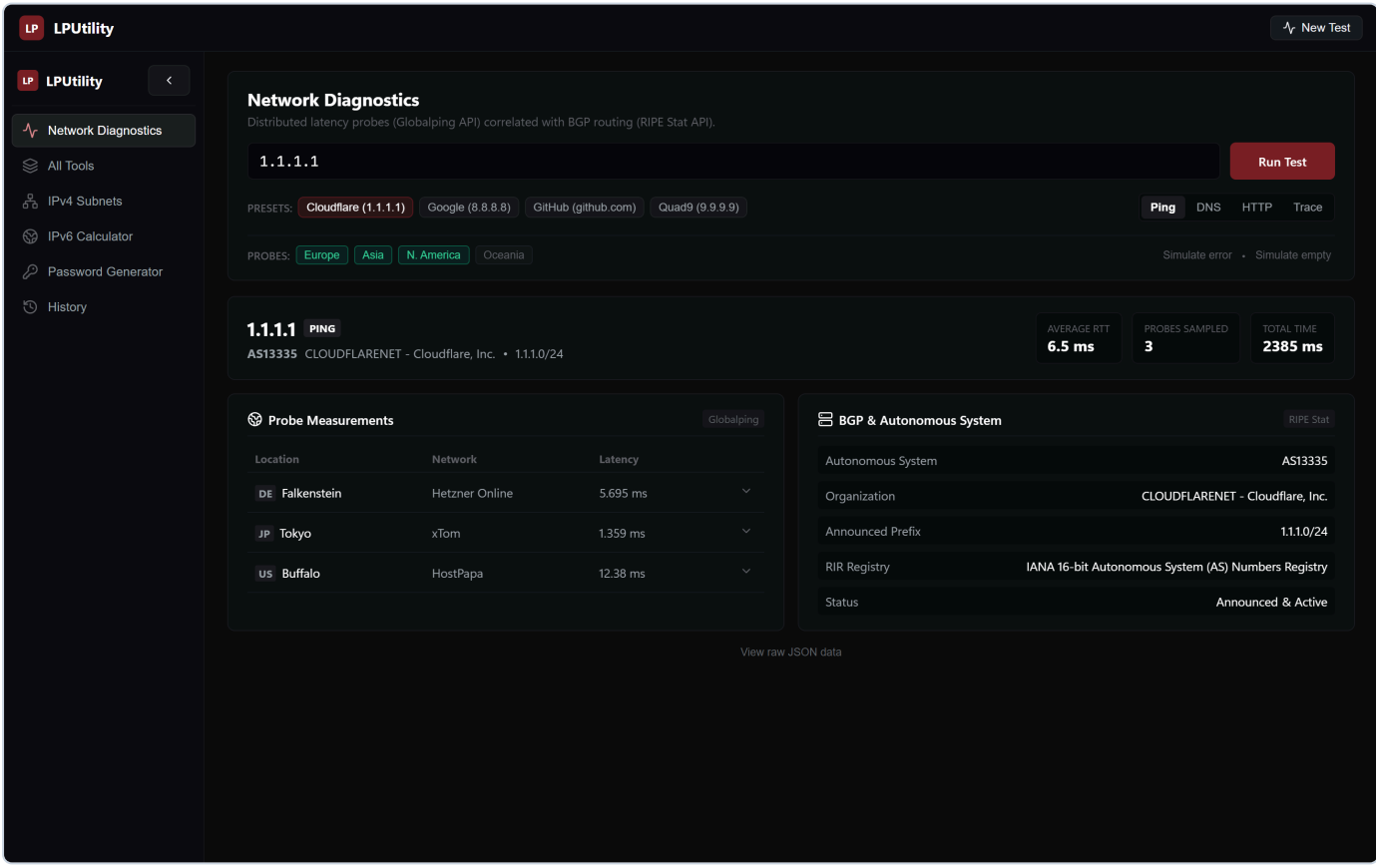


Figure 1: LPUtility Desktop Network Diagnostics displaying live probe results and BGP routing data.

5.2 Tools & Utilities Catalog

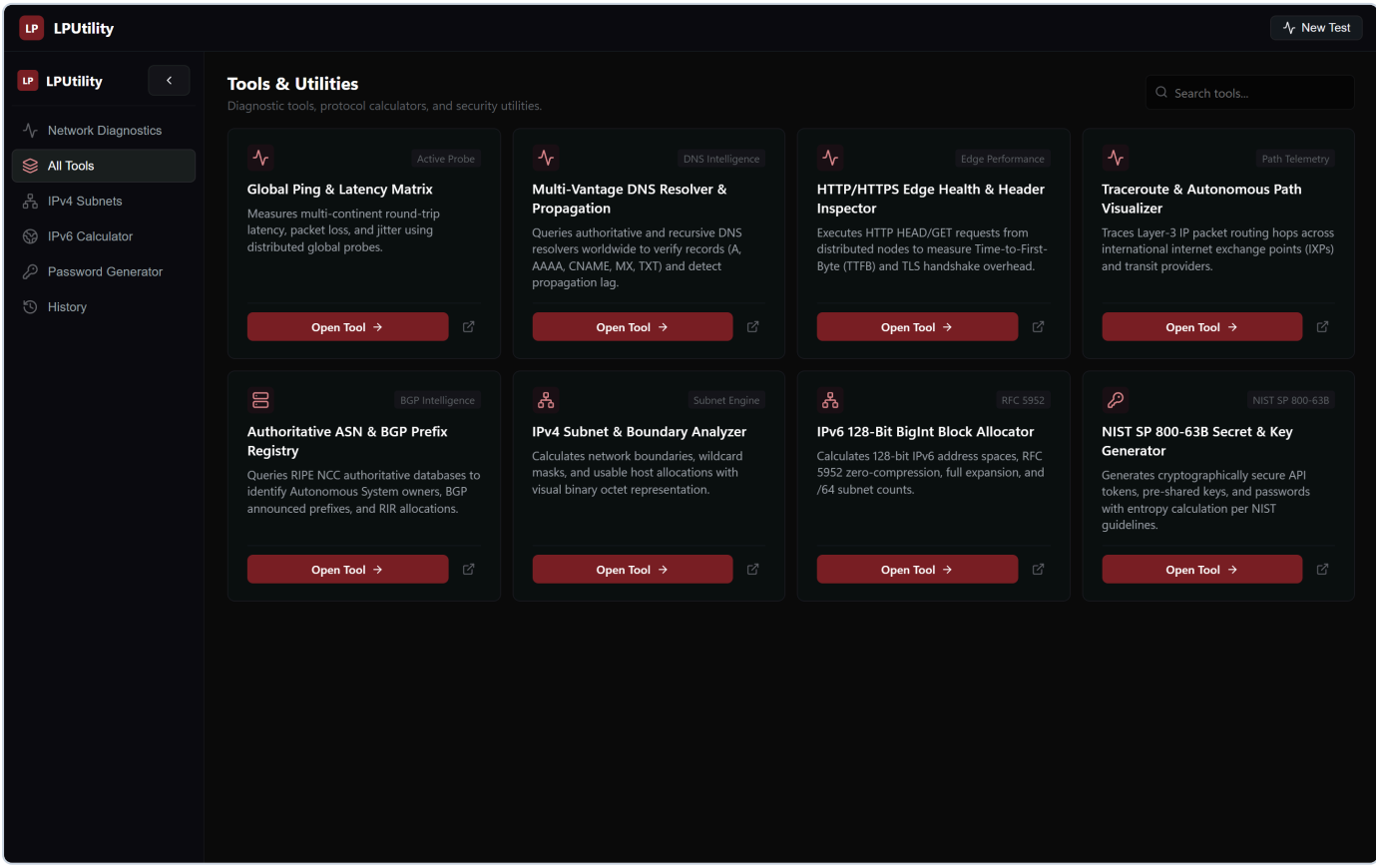


Figure 2: LPUtility Tools Catalog with interactive utilities.

5.3 Protocol Calculators (IPv4 & IPv6)

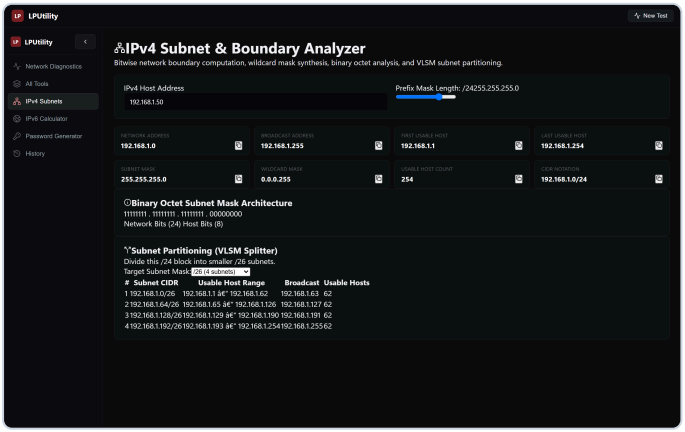


Figure 3: IPv4 Subnet Analyzer.

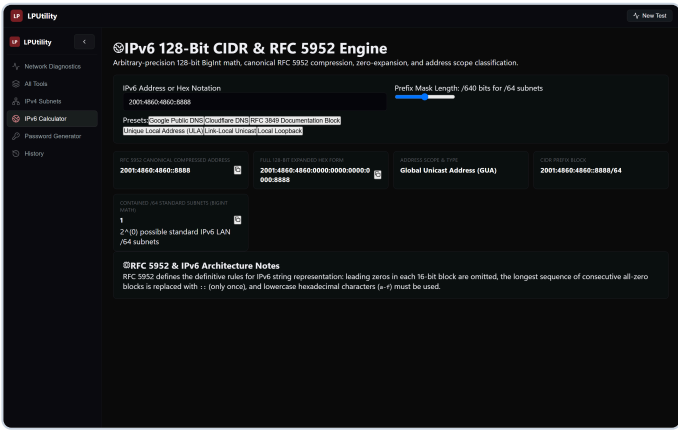


Figure 4: IPv6 128-Bit Calculator.

5.4 Error & Empty State Handling

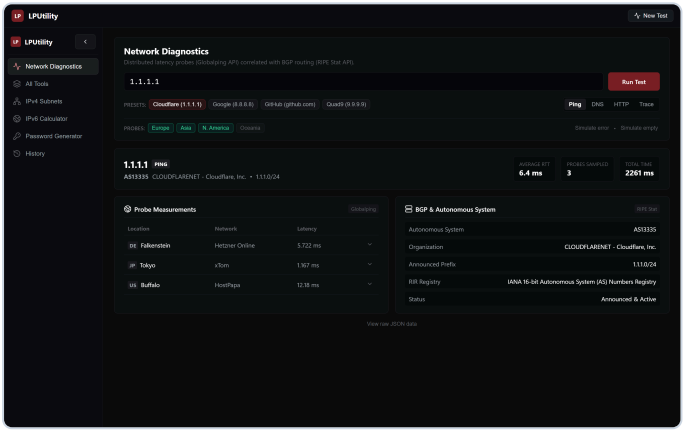


Figure 5: Error state handling with retry action.

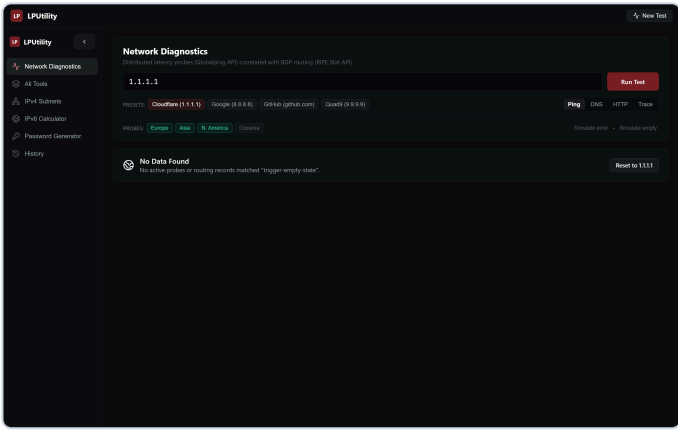


Figure 6: Empty response handling.

5.5 Mobile Responsive Adaptation & Android WebView

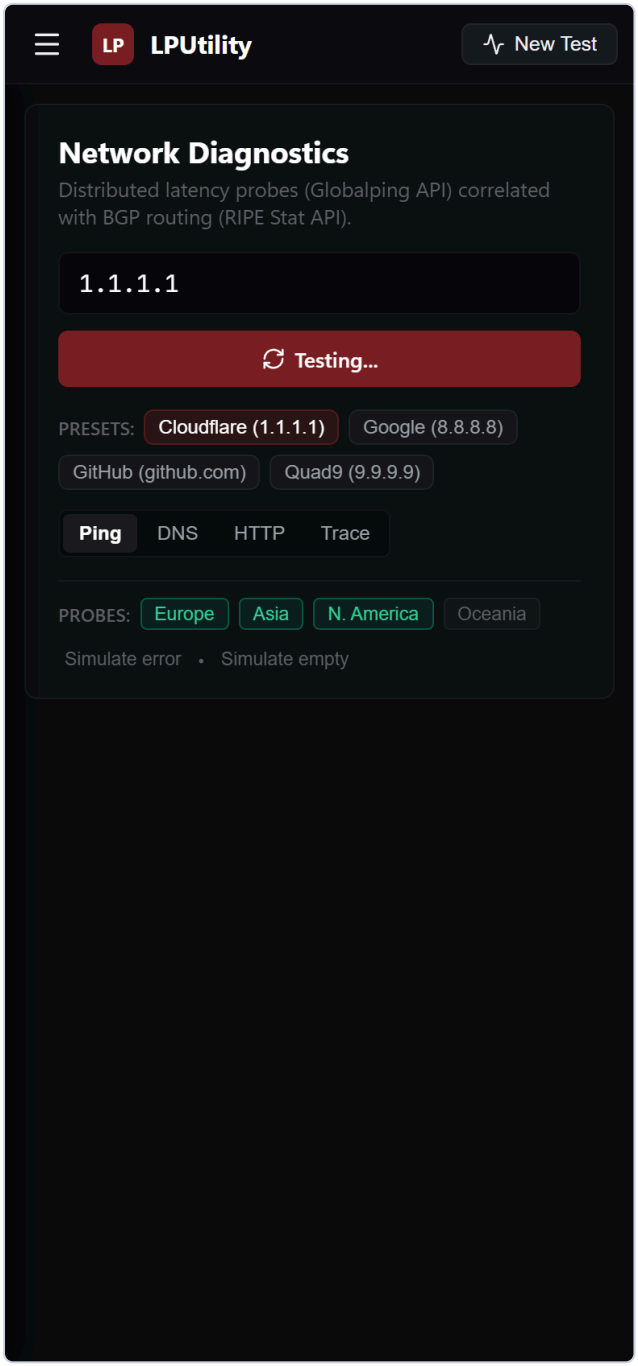


Figure 7: Mobile viewport (390x844).

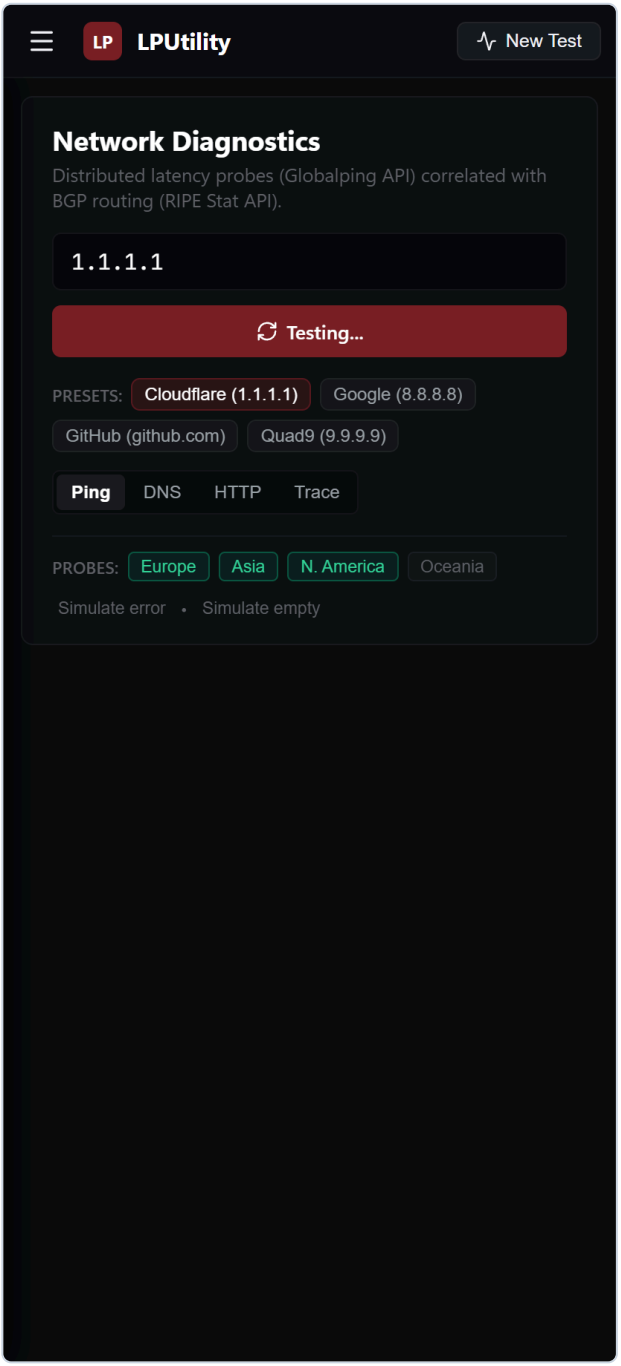


Figure 8: Native Android WebView execution.